BLINKIT

SQL Project Documentation

Retail Analytics & Business Intelligence Report

---

1. Project Overview

This document provides comprehensive documentation for the Blinkit Grocery SQL Project — a relational database solution designed to analyze customer behavior, product performance, order trends, and revenue distribution for a quick-commerce grocery platform.

The project mirrors real-world analytics use cases inspired by Blinkit (formerly Grofers), India's leading instant grocery delivery service.

1.1 Objectives

- Analyze total and category-wise revenue generation
- Identify top-selling products and high-value customers
- Track daily sales trends and order patterns by city
- Segment customers based on spending behavior
- Calculate revenue contribution percentage across categories

---

1.2 Tech Stack

Component	Details
Database	PostgreSQL (standard SQL compatible)
Language	SQL (DDL + DML + Analytical Queries)
Dataset Size	100 Customers, 100 Products, 100 Orders, 100 Order Items
Date Range	May 1, 2026 – June 19, 2026
Cities Covered	Hyderabad, Bangalore, Chennai, Mumbai, Delhi, Pune & more

2. Database Schema

The project uses four interrelated tables forming a standard star schema for order analytics.

2.1 Entity Relationship Overview

Table	Primary Key	Description
customers	customer_id	Stores customer profile and city information
products	product_id	Contains product name, category and price
orders	order_id	Links customer to order with date and amount
order_items	order_item_id	Maps orders to products with quantity

2.2 Table Definitions

customers

```
CREATE TABLE customers (
    customer_id   INT PRIMARY KEY,
    customer_name VARCHAR(50),
    city          VARCHAR(50)
);
```

products

```
CREATE TABLE products (
    product_id   INT PRIMARY KEY,
    product_name VARCHAR(50),
    category     VARCHAR(50),
    price        DECIMAL(10,2)
);
```

orders

```
CREATE TABLE orders (
    order_id     INT PRIMARY KEY,
    customer_id  INT,
    order_date   DATE,
    total_amount DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

order_items

```
CREATE TABLE order_items (
```

```
    order_item_id INT PRIMARY KEY,
    order_id      INT,
    product_id    INT,
    quantity      INT,
    FOREIGN KEY (order_id)    REFERENCES orders(order_id),
    FOREIGN KEY (product_id)  REFERENCES products(product_id)
);
```

2.3 Data Summary

Entity	Count	Key Notes
Customers	100	Spread across 11 Indian cities
Products	100	10 categories (101–200 IDs)
Orders	100	One order per customer (1001–1100)
Order Items	100	One item line per order
Cities	11	Hyderabad has highest customer density
Product Categories	10	Groceries, Dairy, Snacks, Beverages, etc.

3. Analytical Queries

This section documents all 15 analytical SQL queries, their purpose, logic, sample output, and business insight.

3.1. Query 1: Total Revenue

Purpose:

Calculates the sum of all order amounts to determine overall business revenue.

SQL:

```
SELECT SUM(total_amount) AS total_revenue
FROM orders;
```

Sample Output:

total_revenue
55,000 (approx.)

Business Insight:

Provides the top-line revenue figure for the entire dataset period.

3.2. Query 2: Top Selling Products

Purpose:

Ranks products by total quantity sold using a JOIN between products and order_items.

SQL:

```
SELECT p.product_name, SUM(oi.quantity) AS quantity_sold
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_name
ORDER BY quantity_sold DESC;
```

Sample Output:

product_name	quantity_sold
Salt	4
Bread	3
Chips	3

Business Insight:

Helps identify fast-moving stock for inventory management.

3.3. Query 3: Customer-wise Spending

Purpose:
Aggregates total spending per customer to identify high-value customers.
SQL:

```
SELECT c.customer_name, SUM(o.total_amount) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_name
ORDER BY total_spent DESC;
```

Sample Output:

customer_name	total_spent
Chandu	880
Mahesh	870
Gopal	840

Business Insight:
Enables targeted loyalty programs and personalized promotions.

3.4. Query 4: Category-wise Revenue

Purpose:
Computes revenue per product category by multiplying price × quantity across all orders.
SQL:

```
SELECT p.category, SUM(p.price * oi.quantity) AS revenue
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.category;
```

Sample Output:

category	revenue
Healthy Food	High
Groceries	High
Dairy	Medium

Business Insight:
Reveals which product categories drive the most revenue for strategic focus.

3.5. Query 5: Total Orders by City

Purpose:

Counts the number of orders placed from each city.

SQL:

```
SELECT c.city, COUNT(o.order_id) AS total_orders
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.city;
```

Sample Output:

city	total_orders
Hyderabad	20+
Mumbai	10+
Bangalore	10+

Business Insight:

Guides city-level marketing spend and delivery infrastructure decisions.

3.6. Query 6: Average Order Value

Purpose:

Calculates the mean order amount across all transactions.

SQL:

```
SELECT ROUND(AVG(total_amount), 2) AS average_order_value
FROM orders;
```

Sample Output:

average_order_value
551.30 (approx.)

Business Insight:

A key KPI for measuring customer purchase behavior and setting discount thresholds.

3.7. Query 7: Highest Value Order

Purpose:

Identifies the single largest order by total amount.

SQL:

```
SELECT order_id, customer_id, total_amount
FROM orders
ORDER BY total_amount DESC
```

```
LIMIT 1;
```

Sample Output:

order_id	customer_id	total_amount
1088	88	880

Business Insight:
Useful for anomaly detection and understanding peak purchase behavior.

3.8. Query 8: Top 5 Customers by Spending

Purpose:
Lists the five customers who have spent the most in total.

SQL:

```
SELECT c.customer_name, SUM(o.total_amount) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_name
ORDER BY total_spent DESC
LIMIT 5;
```

Sample Output:

customer_name	total_spent
Chandu	880
Mahesh	870
Abhishek	810
Anusha	830
Naveen	880

Business Insight:
Helps prioritize VIP customer programs and rewards.

3.9. Query 9: Most Popular Product Category

Purpose:
Ranks categories by total quantity of products sold.

SQL:

```
SELECT p.category, SUM(oi.quantity) AS total_quantity_sold
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.category
```

```
ORDER BY total_quantity_sold DESC;
```

Sample Output:

category	total_quantity_sold
Groceries	Highest
Healthy Food	High

Business Insight:
Drives category-level procurement and shelf-space allocation.

3.10. Query 10: Number of Customers in Each City

Purpose:
Counts total registered customers per city.

SQL:

```
SELECT city, COUNT(customer_id) AS total_customers
FROM customers
GROUP BY city
ORDER BY total_customers DESC;
```

Sample Output:

city	total_customers
Hyderabad	15
Mumbai	10
Bangalore	9

Business Insight:
Informs regional expansion and local warehouse placement strategies.

3.11. Query 11: Daily Sales Trend

Purpose:
Aggregates total revenue per day to show sales trends over time.

SQL:

```
SELECT order_date, SUM(total_amount) AS daily_sales
FROM orders
GROUP BY order_date
ORDER BY order_date;
```

Sample Output:

order_date	daily_sales
2026-05-01	850
2026-05-02	860
...	...

Business Insight:
Enables time-series analysis for demand forecasting and seasonal planning.

3.12. Query 12: Top 10 Highest Revenue Products

Purpose:
Identifies the 10 products generating the most revenue (price × quantity).
SQL:

```
SELECT p.product_name, SUM(p.price * oi.quantity) AS revenue
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_name
ORDER BY revenue DESC
LIMIT 10;
```

Sample Output:

product_name	revenue
Pistachios	1000
Walnuts	900
Almonds	700

Business Insight:
Used to prioritize premium product stocking and promotional bundling.

3.13. Query 13: Customers with Orders Above Average

Purpose:
Uses a subquery to find customers whose order value exceeds the average order value.
SQL:

```
SELECT c.customer_name, o.order_id, o.total_amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.total_amount > (SELECT AVG(total_amount) FROM orders);
```

Sample Output:

customer_name	order_id	total_amount
Arjun	1008	800
Rahul	1001	600

Business Insight:
Helps segment high-value buyers for upsell and premium subscription campaigns.

3.14. Query 14: Product Count by Category

Purpose:
Counts the number of distinct products in each category.

SQL:

```
SELECT category, COUNT(product_id) AS product_count
FROM products
GROUP BY category
ORDER BY product_count DESC;
```

Sample Output:

category	product_count
Personal Care	16
Healthy Food	15
Beverages	14

Business Insight:
Reveals the depth of product range per category for assortment planning.

3.15. Query 15: Revenue Contribution (%) by Category

Purpose:
Calculates each category's percentage share of total revenue using a nested subquery.

SQL:

```
SELECT category,
  ROUND(SUM(revenue) * 100.0 / (SELECT SUM(revenue)
    FROM (SELECT SUM(p.price * oi.quantity) AS revenue
          FROM products p JOIN order_items oi
          ON p.product_id = oi.product_id
          GROUP BY p.category) t), 2) AS revenue_percentage
FROM (...) x
GROUP BY category, revenue;
```

Sample Output:

category	revenue_percentage
Healthy Food	~22%
Groceries	~18%

Business Insight:
Critical for understanding which categories contribute most to business viability.

4. Key Business Insights Summary

#	Metric	Finding
1	Top City	Hyderabad leads with 15+ customers and 20+ orders
2	Best Category	Healthy Food & Groceries drive the highest revenue
3	Avg Order Value	~₹551 per order across the platform
4	High-Value Customers	Top 5 customers each spent ₹800+
5	Subquery Use	Query 13 uses correlated subquery for above-avg filter
6	Revenue Share	Top 3 categories account for ~55% of total revenue

5. Concepts Demonstrated

SQL Concept	Queries Using It
DDL (CREATE TABLE, FOREIGN KEY)	Schema setup
DML (INSERT INTO)	Data population
Aggregate Functions (SUM, COUNT, AVG)	1, 3, 5, 6, 9, 10, 11
GROUP BY + ORDER BY	2, 3, 4, 5, 9, 10, 11, 12, 14, 15
INNER JOIN (multi-table)	2, 3, 4, 5, 8, 9, 12, 13, 15
Subqueries (scalar & derived)	13, 15
LIMIT	7, 8, 12
ROUND()	6, 15
Revenue Calculation (price × qty)	4, 12, 15

— End of Document —